

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

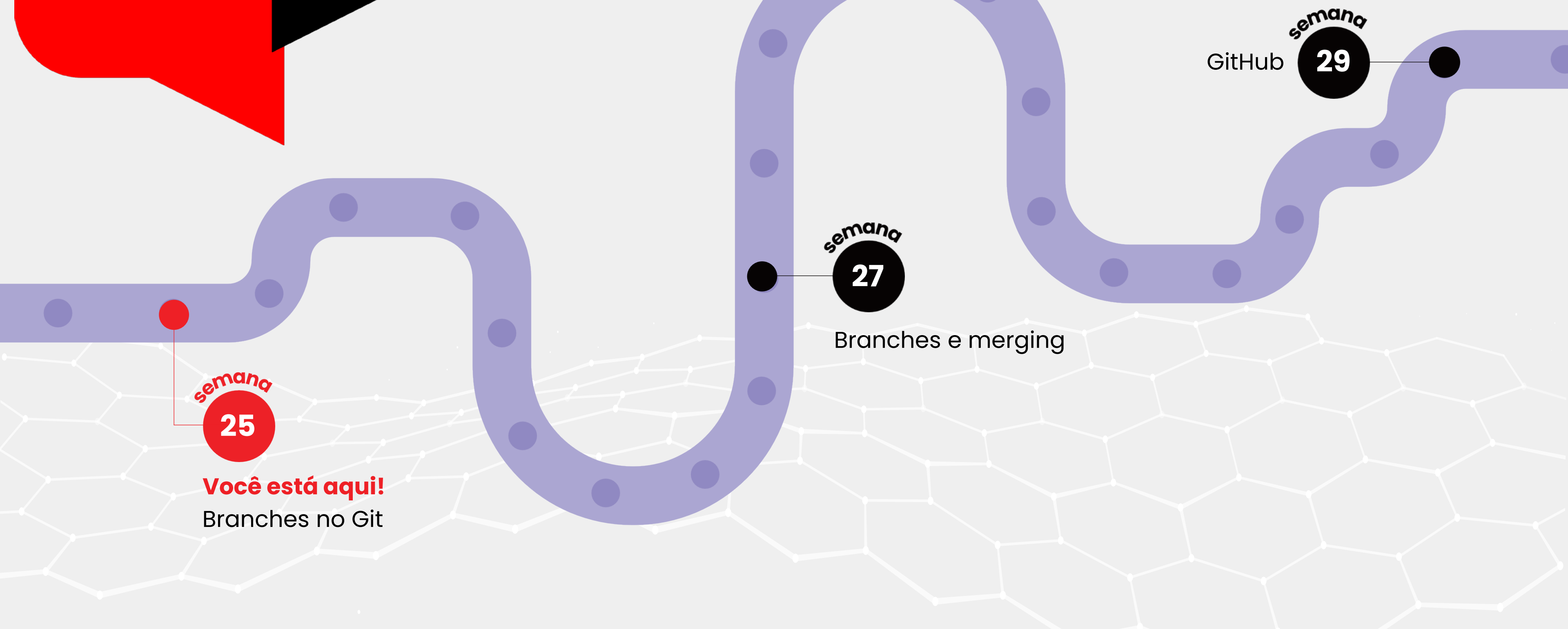
Branches no Git

Fluxos de trabalho e melhores práticas com branches no Git

Código da aula: [DADOS]ANO1C1B4S25A3

Controle de versão
com Git e GitHub

Mapa da Unidade 8 Componente 1



Controle de versão
com Git e GitHub

Mapa da
Unidade 8
Componente 1

Você está aqui!

Branches no Git

**Fluxos de trabalho e melhores
práticas com branches no Git**

Código da aula: [DADOS]ANO1C1B4S25A3

25



Objetivos da aula

- Introduzir e utilizar ferramentas de desenvolvimento de software como Git e GitHub.



Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou internet.



Duração da Aula

50 minutos.



Competências técnicas

- Usar ferramentas de desenvolvimento de software como Git e GitHub;
- Compreender e dominar técnicas de manipulação de dados.



Competências socioemocionais

- Analisar informações de forma crítica e tomar decisões;
- Avaliar a qualidade dos dados e a eficácia dos modelos e das técnicas de análise;
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais, colaborando com colegas, gestores e clientes.

Construindo
o **conceito**

Introdução aos fluxos de trabalho com branches

Os fluxos de trabalho com branches são metodologias que definem como os desenvolvedores devem criar, gerenciar e integrar branches em um projeto. Ajudam, também, a manter a organização e a eficiência no desenvolvimento de software.

Git Flow	GitHub Flow	GitLab Flow
É um modelo de branching que utiliza branches específicos para desenvolvimento, releases e hotfixes. Ele define um processo claro para a criação e integração de novas funcionalidades, preparação de releases e aplicação de correções de bugs.	É um modelo de branching mais simples, focado em um único branch de desenvolvimento (main) e branches de funcionalidades. Cada nova funcionalidade é desenvolvida em um branch separado e integrada ao main por meio de pull requests.	Combina elementos de Git Flow e GitHub Flow, suportando múltiplos ambientes de deploy e integração contínua. Ele permite a criação de branches para desenvolvimento, staging e produção, facilitando o gerenciamento de releases e deploys.

Elaborado especialmente para o curso.

Construindo
o **conceito**

Melhores práticas para gerenciamento de branches

As melhores práticas para gerenciamento de branches incluem:

- ▶ a criação de branches curtos e focados;
- ▶ a integração frequente de mudanças;
- ▶ a utilização de pull requests para revisões de código;
- ▶ a resolução rápida de conflitos.



Tome nota

Essas práticas ajudam a manter o código organizado e a reduzir o risco de problemas.

Construindo o conceito

Exemplos práticos

Exemplo 1: Exemplo de Git Flow

```
import os

# Cria uma pasta de trabalho para o repositório
os.makedirs("my_git_flow_project", exist_ok=True)
%cd my_git_flow_project

# Inicializa o repositório Git
!git init

# Configura o nome e o e-mail do Git
!git config user.name "Seu Nome"
!git config user.email "seu_email@example.com"

# Cria o branch develop e faz o commit inicial
!git checkout -b develop
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

print(add_numbers(2, 3))""")

!git add sum.py
!git commit -m "Initial commit on develop"
```

```
# Cria um branch de feature e adiciona uma nova função
!git checkout -b feature/add_multiply
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

def multiply_numbers(a, b):
    return a * b

print(add_numbers(2, 3))
print(multiply_numbers(3, 4))""")

!git add sum.py
!git commit -m "Added multiply_numbers function in feature/add_multiply"

# Volta para o branch develop e faz o merge
!git checkout develop
!git merge feature/add_multiply

# Cria um branch de release
!git checkout -b release/1.0
!git commit -m "Release 1.0"

# Volta para o branch develop e faz o merge
!git checkout develop
!git merge release/1.0

# Volta para o branch main e faz o merge do release
!git checkout main
!git merge release/1.0

# Mostra o conteúdo final do arquivo após o merge
with open("sum.py", "r") as file:
    print(file.read())
```

Elaborado especialmente
para o curso com a
ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplos práticos

Exemplo 2: Exemplo de GitHub Flow

```
import os

# Cria uma pasta de trabalho para o repositório
os.makedirs("my_github_flow_project", exist_ok=True)
%cd my_github_flow_project

# Inicializa o repositório Git
!git init

# Configura o nome e o e-mail do Git
!git config user.name "Seu Nome"
!git config user.email "seu_email@example.com"

# Cria o arquivo sum.py e faz o commit inicial
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

print(add_numbers(2, 3))""")

!git add sum.py
!git commit -m "Initial commit"
```

```
# Cria um branch de feature e adiciona uma nova função
!git checkout -b feature/add_subtract
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

def subtract_numbers(a, b):
    return a - b

print(add_numbers(2, 3))
print(subtract_numbers(5, 2))""")

!git add sum.py
!git commit -m "Added subtract_numbers function in feature/add_subtract"

# Volta para o branch main e faz o merge do branch de feature
!git checkout main
!git merge feature/add_subtract

# Mostra o conteúdo final do arquivo após o merge
with open("sum.py", "r") as file:
    print(file.read())
```

Elaborado especialmente
para o curso com a
ferramenta Jupyter Notebook.

Construindo o conceito

Exemplos práticos

Exemplo 3: Exemplo de GitLab Flow

```
import os

# Cria uma pasta de trabalho para o repositório
os.makedirs("my_gitlab_flow_project", exist_ok=True)
%cd my_gitlab_flow_project

# Inicializa o repositório Git
!git init

# Configura o nome e o e-mail do Git
!git config user.name "Seu Nome"
!git config user.email "seu_email@example.com"

# Cria o arquivo sum.py e faz o commit inicial no branch develop
!git checkout -b develop
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

print(add_numbers(2, 3))""")

!git add sum.py
!git commit -m "Initial commit on develop"
```

```
# Cria um branch de feature e adiciona uma nova função
!git checkout -b feature/add_divide
with open("sum.py", "w") as file:
    file.write("""def add_numbers(a, b):
    return a + b

def divide_numbers(a, b):
    return a / b

print(add_numbers(2, 3))
print(divide_numbers(6, 2))""")

!git add sum.py
!git commit -m "Added divide_numbers function in feature/add_divide"

# Volta para o branch develop e faz o merge
!git checkout develop
!git merge feature/add_divide

# Cria um branch de staging e faz o merge do develop
!git checkout -b staging
!git merge develop

# Volta para o branch develop e faz o merge do staging
!git checkout develop
!git merge staging

# Volta para o branch main e faz o merge do staging
!git checkout main
!git merge staging

# Mostra o conteúdo final do arquivo após o merge
with open("sum.py", "r") as file:
    print(file.read())
```

Elaborado especialmente
para o curso com a
ferramenta Jupyter Notebook.

Colocando em **prática**

Exercício: Criar um repositório Git



20 minutos



Em duplas

1

Configurar o repositório

Criem um diretório de trabalho chamado `simple_project`.
Inicializem um repositório Git no diretório.
Configurem seu nome de usuário e e-mail no Git.

2

Criar o arquivo inicial

Criem um arquivo `calc.py` com uma função `add_numbers`.
Façam o commit inicial no branch `main`.

3

Criar um branch de funcionalidade

Criem um novo branch chamado `feature/subtract`.
Adicionem uma função `subtract_numbers` ao arquivo `calc.py`.
Façam um commit no branch `feature/subtract`.

4

Integrar a funcionalidade

Voltem para o branch `main`.
Façam o merge do branch `feature/subtract` no `main`.

Colocando
em **prática**

Exercício: criar um repositório Git

Entrega

1. Arquivo calc.py Final
 - O arquivo deve incluir as funções `add_numbers` e `subtract_numbers`.
2. Histórico de commits
 - Inclua uma captura de tela ou uma lista do histórico de commits mostrando os commits realizados e o merge.



20 minutos



Em duplas

Ser
sempre +

Situação

Você está trabalhando como líder em um projeto de software com uma equipe de cinco desenvolvedores.

Cada membro da equipe é responsável por diferentes funcionalidades do projeto.

Recentemente, você notou que alguns colegas estão tendo dificuldades para entender e usar corretamente o fluxo de trabalho do Git, resultando em conflitos de merge frequentes e frustração.

Situação fictícia elaborada especialmente para o curso.

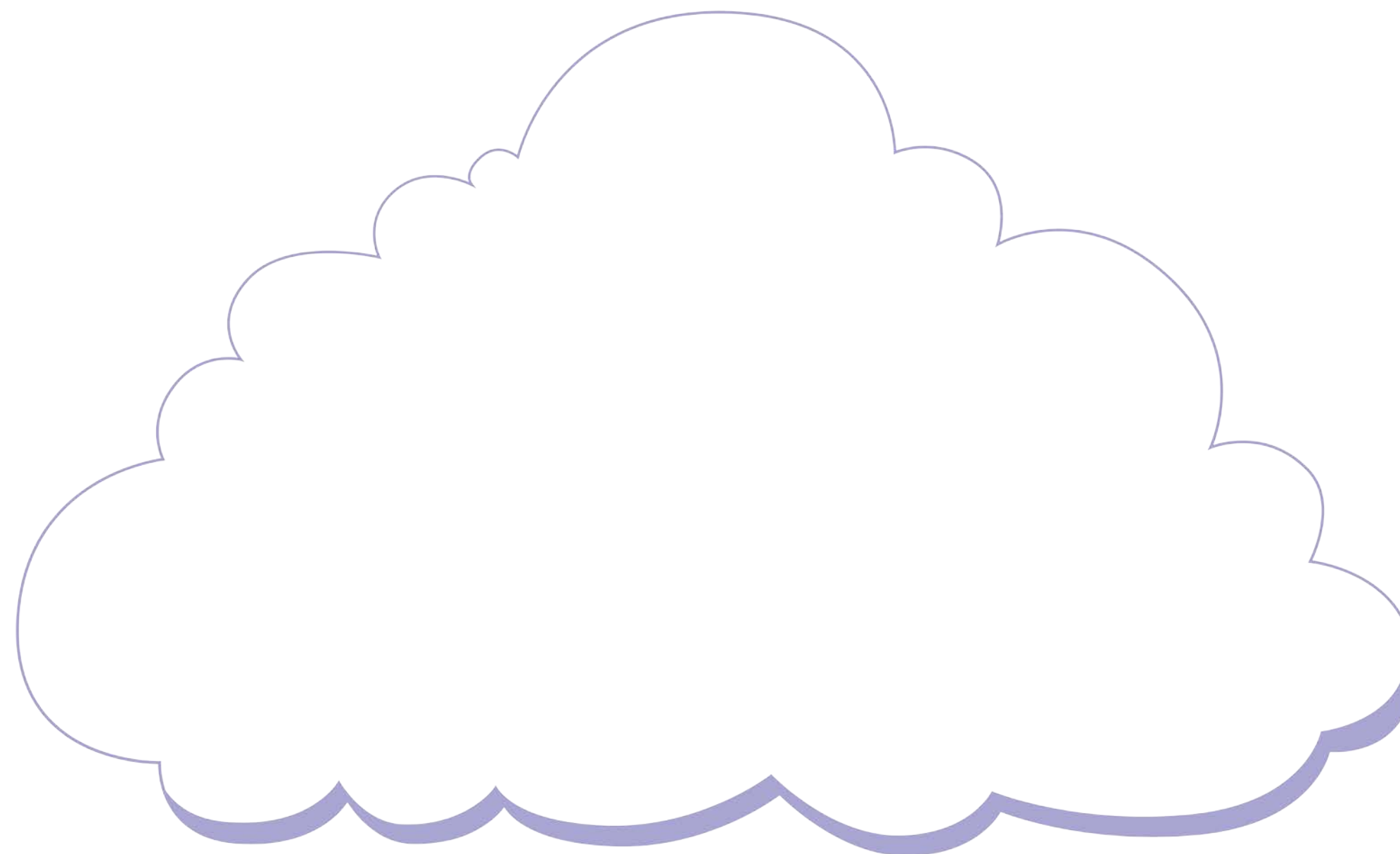
Ser
sempre +

Ação

Perguntas para reflexão e ação:

1. Como você pode promover um ambiente de apoio no qual todos os membros da equipe se sintam à vontade para pedir ajuda e compartilhar suas dificuldades?
2. Quais estratégias você pode sugerir para reduzir os conflitos de merge e melhorar o uso do Git pela equipe?
3. Como você pode ajudar a equipe a desenvolver habilidades socioemocionais, como empatia e resiliência, ao lidar com os desafios técnicos do projeto?

Nuvem de palavras



© Getty Images

O que nós
**aprendemos
hoje?**



© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** Git Flow: um modelo de branching estruturado que usa diferentes branches para desenvolvimento (develop), releases (release) e hotfixes. Facilita a gestão de novas funcionalidades, correções de bugs e preparação de releases.
- 2** GitHub Flow: um modelo de branching mais simples, focado em um único branch de desenvolvimento (main) e branches de funcionalidades, com integração por meio de pull requests.
- 3** GitLab Flow: combina elementos de Git Flow e GitHub Flow, suportando múltiplos ambientes de deploy e integração contínua. Utiliza branches para desenvolvimento, staging e produção, facilitando o gerenciamento de releases e deploys.

Saiba mais

Por que se perder no labirinto do Git quando você pode dominar o controle de versão com esse curso completo?

Domine branches, merge, rebase, tags, releases e muito mais!

ALURA. *Git e GitHub*: dominando controle de versão de código.

Disponível em: <https://cursos.alura.com.br/course/git-github-dominando-controle-versao-codigo>.

Acesso em: 12 jul. 2024.

Referências da aula

BELL, P.; BEER, B. *Introdução ao GitHub*: um guia que não é técnico. São Paulo: Novatec, 2014.

GITHUB. *Documentação de introdução ao GitHub*, [s.d.]. Disponível em: <https://docs.github.com/pt/get-started>. Acesso em: 12 jul. 2024.

Identidade visual: imagens © Getty Images.

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**